SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	M. Niharika	**Reg. No.:**	192472392
Section / Batch:	CSE-AI (2024)	**Date:**	09/01/26

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:		100 Marks

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I M.Nihanika (192472392) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Nihanika

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
					Overall Total (100)	80

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Applications: Smart Attendance System

A smart attendance system scans each student's record exactly once to mark attendance and generate reports. If there are n students, each record is processed only one time. Therefore, the number of operations increases directly with the number of students. This is called linear time complexity, where the execution time grows proportionally as the input size increases.

Time Complexity:

- Number of student records = n
- Each record is scanned once
- Total operations = n

Therefore,

Time Complexity = O(n)

Justification: Since every student is processed exactly once and no repeated scanning is performed, the running time increases linearly with the number of students. Hence, the asymptotic time complexity is O(n).

Space Complexity: The system updates attendance directly in the existing database and requires only a few temporary variables such as counters or flags.

Space Complexity = O(1)

This means the extra memory required remains constant

regardless of the number of students.

142) Application : Rank Comparison System.

A university ranking system compares each student's performance with every other student to determine the overall ranking. This requires comparing every possible pair of students, which results in nested loops. As the number of students increases, the number of comparisons increases rapidly.

Time Complexity:

- Outer loop executes n times
- Inner loop also executes n times
- Total Comparisons = $n \times n = n^2$

Therefore,

Time Complexity = $O(n^2)$

Justification: Since each student is compared with every other student, the number of operations grows quadratically with the input size. Therefore, the algorithm has $O(n^2)$ time complexity.

Space Complexity: If the algorithm stores all comparison results in a matrix, it requires $O(n^2)$ memory. If only temporary variables are used without storing comparisons, the extra memory required is $O(1)$.

Application: Password Lookup System.

A password lookup system uses binary search to locate user credentials in a sorted database. Binary search repeatedly divides the search space in to two equal halves until the required password is found or the search ends. Because the search area becomes half at every step. the number of operations increases very slowly even for large databases.

Time Complexity:

At each Comparison:

- Remaining elements become n/2
- Then n/4
- Then n/8
- And so on

Therefore,

Time Complexity = $O(\log n)$

Justification: Since the search space is reduced by half after every Comparision, the total number of Comparisons is proportional to $\log_2 n$, giving $O(\log n)$ Complexity.

Space Complexity:

- Iterative Binary Search: $O(1)$
- Recursive Binary Search: $O(\log n)$ because recursive function calls occupy stack memory.

Space Complexity = $O(n)$

144) Application : Document Processing System.

The recurrence relation is: $T(n) = T(n-1) + 1$

This indicates that one document is processed during each recursive step.

Substitution Method:

Expand the recurrence:

$$T(n) = T(n-1) + 1$$
$$= T(n-2) + 1 + 1$$
$$= T(n-3) + 1 + 1 + 1$$
$$= T(n-4) + 4$$
$$\vdots$$
$$= T(1) + (n-1)$$

Assuming

$$T(1) = 1$$

Then,

$$T(n) = 1 + (n-1)$$
$$T(n) = n$$

Therefore,

Time Complexity = $O(n)$

Space Complexity :-

Each recursive call is stored in the function call stack.

Space Complexity = $O(n)$

Application : Streaming Data System.

The recurrence relation is : $T(n) = 2T(n/2) + n$

This recurrence follows the divide-and-Conquer strategy, where the problem is divided in to two equal parts, each solved recursively, and then combined in linear time.

Applying the Master Theorem:

General form

$$T(n) = aT(n/b) + f(n)$$

Here,

- $a = 2$
- $b = 2$
- $f(n) = n$

Calculate : $n^{\wedge}(\log_2 2)$

Since,

$$f(n) = O(n^{\wedge}(\log_2 2))$$

This satisfies Case 2 of the Master Theorem.

Therefore,

Time Complexity = $O(n \log n)$

Space Complexity :-

The recursion depth is $\log n$, requiring

Space Complexity - $O(\log n)$

Additional memory may also be required for buffering or temporary storage depending on the implementation.